

Contents

1	Classical ML	2
1.1	Linear Regression	2
1.2	Logistic Regression	2
1.3	Perceptron	2
1.4	SVMs	2
2	Deep Learning	2
2.1	Neural Networks	3
2.2	CNNs	4
2.3	RNNs	5

1 Classical ML

1.1 Linear Regression

Linear Regression is one of the most fundamental algorithms in machine learning. It is used to model the relationship between a dependent variable and one or more independent variables. The goal of linear regression is to find the best-fitting line (or hyperplane in higher dimensions) that minimizes the sum of squared differences between the observed values and the predicted values.

A simple example is the line of best fit, $\hat{y}_i = mx_i + b$. If say, we wanted to represent how much the prediction deviates from the true value y_i , we can consider minimizing the magnitude of the vector

$$\begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \\ \vdots \\ \hat{y}_n - y_n \end{bmatrix}$$

In that case, we've used the squared error as the loss function. Notice that the predicted value $\hat{y}_i$ is just the dot product between the vectors $\mathbf{w} = (m, b)$ and $\mathbf{x}_i = (x_i, 1)$. Then we can represent this as a matrix equation

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

or $\mathbf{M}\beta = \hat{\mathbf{y}}$. The goal then is to minimize $\mathbf{M}\beta - \mathbf{y}$ which actually has a closed form solution

$$\beta = (\mathbf{M}^T \mathbf{M})^{-1} \mathbf{M}^T \mathbf{y}$$

Linear regression does not imply linear relation. The $\mathbf{M}$ matrix can have any term in it like

$$\begin{bmatrix} \cos(x_1) & e^{x_1} & x_1 \\ \cos(x_2) & e^{x_2} & x_2 \\ \vdots & \vdots & \vdots \\ \cos(x_n) & e^{x_n} & x_n \end{bmatrix}$$

Having products of features is also valid, however one must actually realize these features were relevant to begin with. If we didn't know, for example, our data followed an exponential distribution, we might not have thought to include e^{x_i} as a feature. This works best on data that is ideally uncorrelated across features, and we strongly know in advance the form of the relation.

1.2 Logistic Regression

1.3 Perceptron

1.4 SVMs

2 Deep Learning

The need for more expressive machine learning models at the cost of interpretability underlies the development of deep learning models. Classical algorithms have high interpretability, rigorous error

bounds, and are computationally efficient. But as seen above, we must diligently select features and have a strong understanding of the underlying behavior in the data to effectively use them, which can be a hopelessly complicated task in the case of, for example, language processing.

Very loosely speaking, we can quantify the "expressiveness" of a model by the number of points it can "shatter." This means that given a set of points, the model can be trained to perfectly classify them in any way. For example, a linear model can shatter 3 points. Given any 3 points we can find a line that perfectly classifies them. Many classical algorithms have low or finite shattering

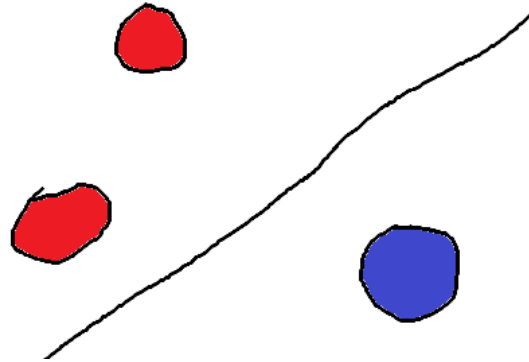


Figure 1: A linear model can only shatter 3 points.

capacity. This is very good not just for interpretability, but because we can actually put rigorous error bounds on how the model generalizes to unseen test data using VC Generalization Bounds. For very complicated tasks however, we would want more expressiveness but this usually entails a model with infinite VC dimension (finite shattering capacity is actually very restrictive), which means we lose PAC Learning and generalization bounds.

2.1 Neural Networks

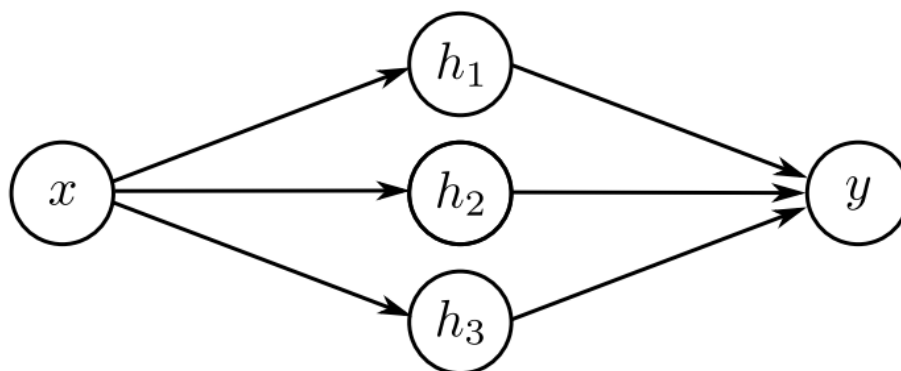


Figure 2: Shallow Neural Network.

The basic unit of a neural network is a perceptron. Each perceptron is equipped with a weight

vector $\mathbf{w}$ and a bias b . An input vector $\mathbf{x}$ is fed into the perceptron, which computes the dot product $\mathbf{w} \cdot \mathbf{x} + b$. Finally, the perceptron passes this through an activation function before sending the output further through the network. The activation function is incredibly important because it allows the network to learn nonlinear behavior. If instead we just sent the output, we would simply have recreated linear regression! A common choice of activation is the rectified linear unit

$$\text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

A famous result is the universal approximation theorem. ANY continuous function can be approximated to any precision using only a shallow neural network (only 1 layer). So theoretically a neural network with 1 layer is enough for any task.

However, for complex tasks, the number of hidden units in the single layer gets prohibitively large. It's actually better to instead of increasing the number of units, to construct more layers. Expressivity grows polynomially with hidden units per layer but exponentially with the number of hidden layers (careful, this measure of expressivity is not the same as VC dimension, but the number of regions the network can partition in input space).

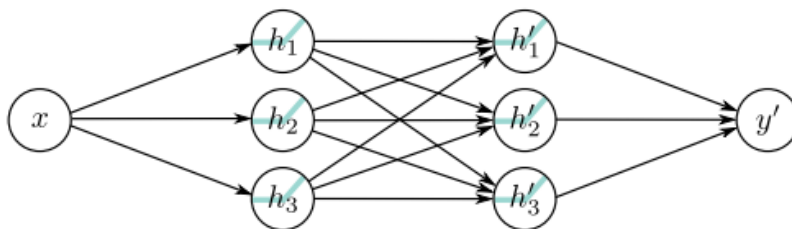


Figure 4.4 Neural network with one input, one output, and two hidden layers, each containing three hidden units.

Pure neural networks have largely been replaced by later architectures. Many more advanced models are actually just subsets of these neural networks but with specific structures more tailored to the task.

2.2 CNNs

Convolutional neural networks were originally designed to classify image data, so many of the design choices were influenced by symmetries and structure of image data.

Firstly, the model's interpretation of an image should be invariant under simple transformations such as rotation or translation. For example, an image of a cat should still be classified as a cat even if we shift or rotate it slightly. Secondly, image data is extremely high dimensional. A typical image with 224×224 RGB values has over 100,000 dimensions. A shallow network with only 2 layers would require over $100,000^2 = 10$ billion parameters. Thirdly, nearby pixels are highly related to each other. If we have a pixel of a cat's nose, the surrounding pixels are highly likely to be very similar.

2.3 RNNs

A recurrent neural network is designed to handle sequential data. The idea is that instead of giving it all the data at once, we feed it one element at a time. Imagine our input is the sentence

"I like football."

In a traditional feed forward neural network, we might train a specific model built to process inputs of sentences of that are 4 words long (and predict whatever, it doesn't matter at the moment). Then each word in the sentence; "I", "like," ... is a separate vector to be input in the neural network with its own sets of weights and biases. However this design is poor for tasks like this for two reasons. First, sentences are usually of variable length, meaning this architecture is not able to generalize. You would have to train a new neural network for every possible sentence length.

Second, there is not enough distinction between say the 9th word in a sentence and a 10th word in a sentence to warrant a separate set of weights and biases to train for them, even less between the 29th and 30th word in a sentence. In fact, there is much statistical information that can be shared between these two words. Thus, it might make more sense to reuse the same weights and bias for each word in the sentence. Take the sentence:

"Football is a fun sport."

The word "football" should have the same meaning whether its the first or last position. Having to make the first and last inputs both learn the meaning of the word "football" is a waste of information that could've been shared. These two reasons are the motivation behind the need to process data sequentially

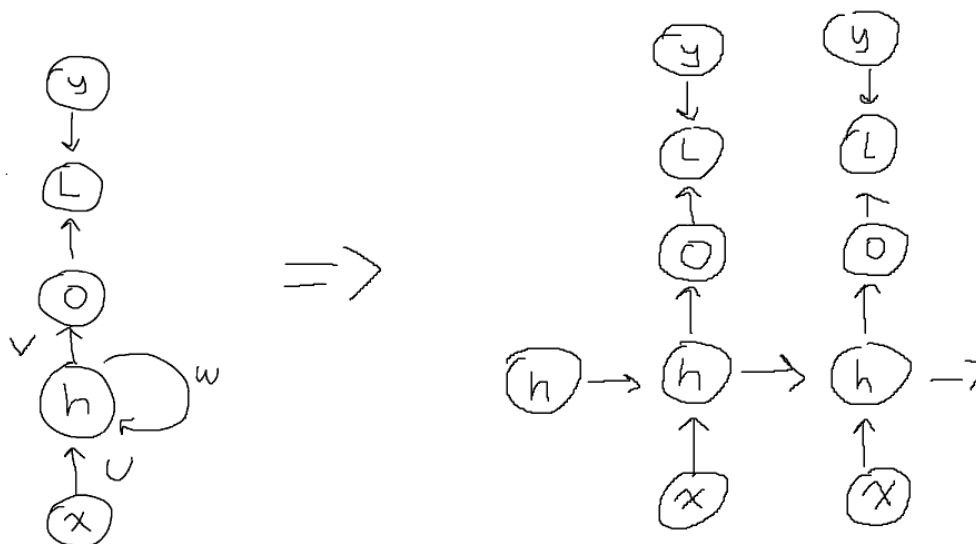


Figure 3: Example RNN Architecture.

An example RNN is shown above. At each instance in time i , this guy takes in an input vector $\mathbf{x}_i$ and then spits out an output o_i . For example, you could imagine this as a device to predict the

weather. Given the current temperature, humidity etc, predict whether its sunny, rainy, etc. A model that is trained on all the data at one time might learn to hardcode the weather, associating it with time dependence instead of actually using the features. An rnn takes this into account, and is forced to learn from the current "state" $\mathbf{h}_i$ and current features $\mathbf{x}_i$.

Additionally, notice that the same weights and biases, the transformation $\mathbf{V}, \mathbf{W}, \mathbf{U}$ are being reused every time. This is in line with our motivation because how we interpret for example, weather from humidity and temperature should not depend on the day it is. In this way, our RNN is forced to learn the features instead of the pattern in the training data.

Aside: this is not the only architecture that exists, one could continuously accumulate the hidden states and only return an output at the very end (which would be more applicable to our situation).

You might have heard of "GRU" or "LSTM" before. These introduce the idea of **gating** to solve the vanishing gradient problem. Over long sequences, the gradient shrinks to 0 making training extremely slow. The idea then is to introduce gates that controls how much information the model should keep from previous timesteps. LSTMs were the first to introduce gating, with the forget, output, and input gates. The forget gate is most important, as it rewrites the current hidden state to drop past information that is no longer relevant. The gate is just a sigmoid

$$f = \sigma(\mathbf{W}_f[h_t, x_t])$$

which then multiplies the current hidden state to drop information

$$h_t = f \odot h_t$$

The gated recurrent unit (GRU) uses only two gates, the reset and update gate. After the hidden layer is updated first, the update gate determines how much of h_t to keep while the reset gate determines how much of the previous state to use.